Unit 6 : Basic Network Management

6.1 IP address configuration: settings in a network. TCP/IP Network address, TCP/IP Configuration files, Network Interfaces and Routes: ifconfig, route, ping, netstat, tcpdump commands.
6.2 DHCP Configuration: Configuring DHCP Client and Server, Dynamic Address, Fixed Addresses.
6.3 NIS, NFS, SAMBA introduction.
6.4 Firewall and Internet Security: Limiting Network Services, Designing Firewall.
6.5 Network Intrusion Detection: Host based Intrusion Detection Software using tripwire any relevant utility.

6.1 A : IP address configuration: TCP/IP Network address

TCP:

TCP or the **Transmission Control Protocol**, which belongs in the Transport Layer of the Internet Protocol Suit, assures reliability and the ordered delivery of information (in the form of byte streams) from one computer to another. Most of the Internet applications that require reliable and secure data transferring such as World Wide Web, E-mail, peer-to-peer file Sharing, Streaming media applications and other file transferring services, uses TCP for transmission and communication purposes.

IP:

IP or the **Internet Protocol** is the basic protocol that makes up the Internet, as it is responsible for the addressing hosts (computers) and transportation of data packets between hosts, through a packet switched internetwork. Residing on the Internet Layer of Internet Protocol Suite, IP only carries out the task of delivering packets of data (Data grams) from one host to another, depending on the host addresses; therefore, is considered unreliable, as Data Packets send through Internet using IP can be lost, corrupted or delivered in an unordered manner.

IP address:

An IP address, short for **Internet Protocol address**, is an identifying number for network hardware connected to a network. Having an IP address allows a device to communicate with other devices over an IP-based network like the internet.

IP Versions (IPv4 vs IPv6)

IPv4: The way IPv4 addresses are constructed means it's able to provide over **4 billion** unique IP addresses (232). While this is a large number of addresses, it's not enough for the modern world with all the different devices used on the internet.

Example:

IPv4 addresses look like this:

```
151.101.65.121
```

IPv6: IPv6 supports **340 trillion**, trillion, trillion addresses (2128). That's 340 with 12 zeros! This means every person on earth could connect billions of devices to the internet.

Example:

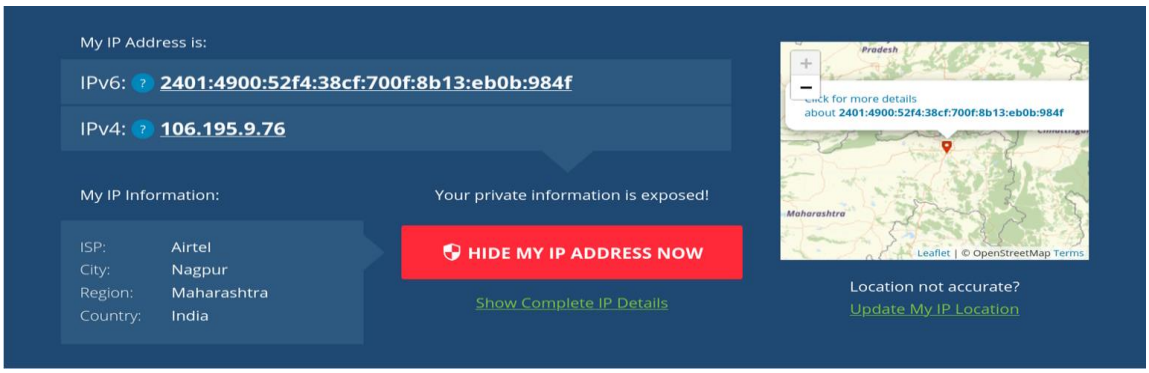
IPv6 addresses look like this:

```
2001:4860:4860::8844
```

Know your IP address

There are several ways to find a router's public IP address, but sites like IP Chicken, WhatsMyIP.org, WhatIsMyIPAddress.com or icanhazip.com make this easy. These sites work on any network-connected device (such as a smartphone, iPod, laptop, desktop, or tablet) that supports a web browser.

Example:



Here is my IP address using WhatIsMyIPAddress.com.

6.1 B : IP address configuration: TCP/IP

Configuration files

You can configure TCP/IP networking when you install Linux. If you want to manage the network on a Linux system effectively, however, you need to become familiar with the TCP/IP configuration files so that you can edit the files, if necessary. (If you want to check whether the name servers are specified correctly, for example, you have to know about the **/etc/resolv.conf** file, which stores the IP addresses of name servers.)

The table below summarizes the basic TCP/IP configuration files.

This File	Contains the Following
`/etc/hosts`	IP addresses and host names for your local network as well as any other systems that you access often
`/etc/networks`	Names and IP addresses of networks
`/etc/host.conf`	Instructions on how to translate host names into IP addresses
`/etc/resolv.conf`	IP addresses of name servers
`/etc/hosts.allow`	Instructions on which systems can access Internet services on your system
`/etc/hosts.deny`	Instructions on which systems must be denied access to Internet services on your system
`/etc/nsswitch.conf`	Instructions on how to translate host names into IP addresses

Installation of nano text editor

You can view as well as and configure these files using any text editor. Here I am using nano text editor. If you don't have you can install it using following command.

```
sudo apt-get install nano
```

Open and Edit files using nano

1) Open File

You can open any file in nano text editor using following command.

```
sudo nano filename
```

2) Save File

You can save your opened file in nano text editor using **Ctrl + O**

3) Save File and Exit

You can save file and exit at the same time from nano text editor using

Ctrl + X and **Y** to confirm.

TCP/IP Configuration files:

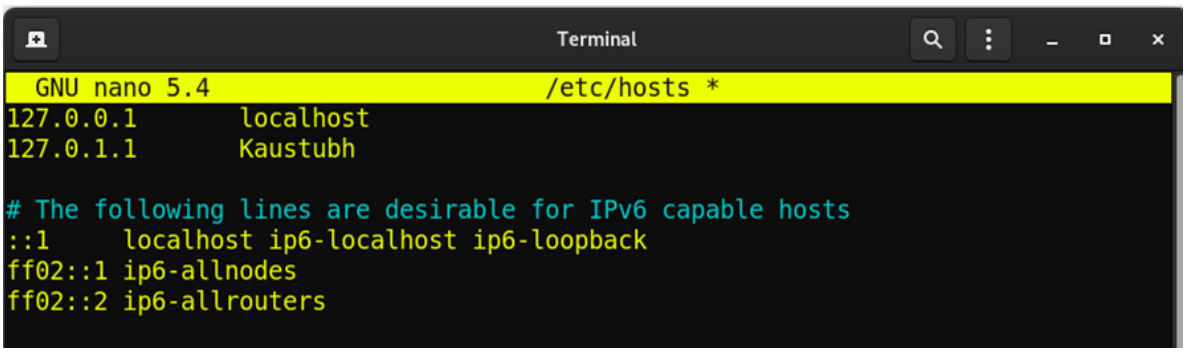
1) /etc/hosts on a Linux system

The /etc/hosts text file contains a list of IP addresses and host names for your local network. In the absence of a name server, any network program on your system consults this file to determine the IP address that corresponds to a host name. Think of `/etc/hosts` as the local phone directory where you can look up the IP address (instead of a phone number) for a local host.

Here is way to open this file in nano text editor which we have installed previously.

```
sudo nano /etc/hosts
```

Here is the output of nano text editor. Now you can edit and save this file.

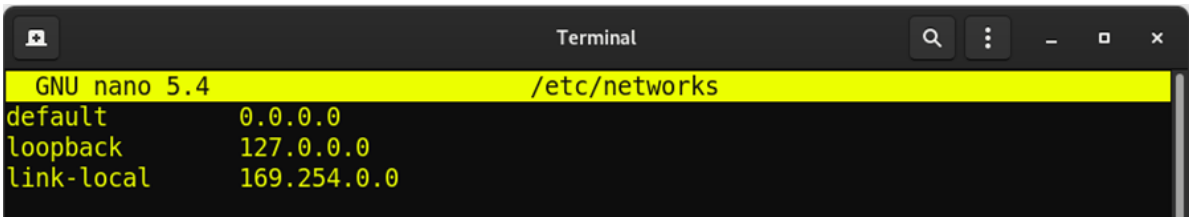


2) /etc/networks on a Linux system

/etc/networks is another text file that contains the names and IP addresses of networks. These network names are commonly used in the routing command

(/sbin/route) to specify a network by its name instead of by its IP address.

Don't be alarmed if your Linux PC doesn't have the /etc/networks file. Your TCP/IP network works fine without this file. In fact, the Linux installer doesn't create a /etc/networks file.

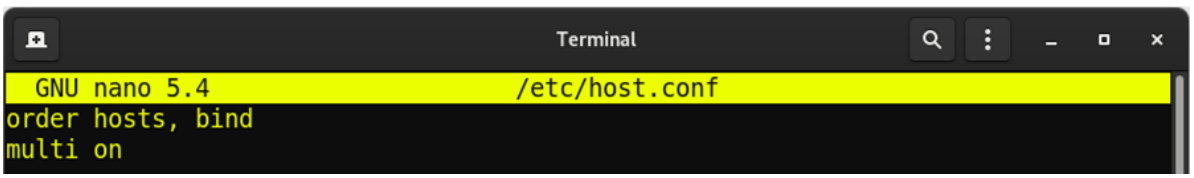


3) /etc/host.conf on a Linux system

Linux uses a special library (collection of computer code) called the resolver to obtain the IP address that corresponds to a host name. The /etc/host.conf file specifies how names are resolved (that is, how the name gets converted to a numeric IP address). A typical **/etc/host.conf** file might contain the following lines:

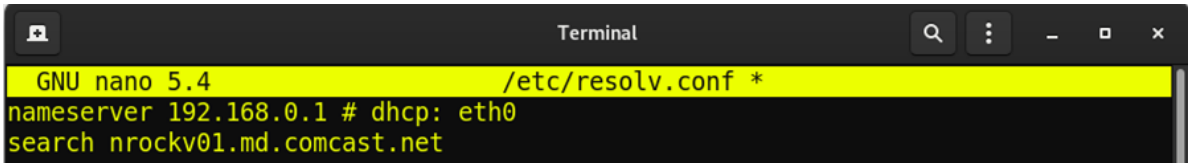
The entries in the **/etc/host.conf** file tell the resolver what services to use (and in which order) to resolve names.

The order option indicates the order of services (in recent distributions, the **nsswitch.conf** file). The sample entry tells the resolver to first consult the **/etc/hosts** file and then check the name server to resolve a name.



4) /etc/resolv.conf on a Linux system

The /etc/resolv.conf file is another text file used by the resolver — the library that determines the IP address for a host name. Here's a sample **/etc/resolv.conf** file:

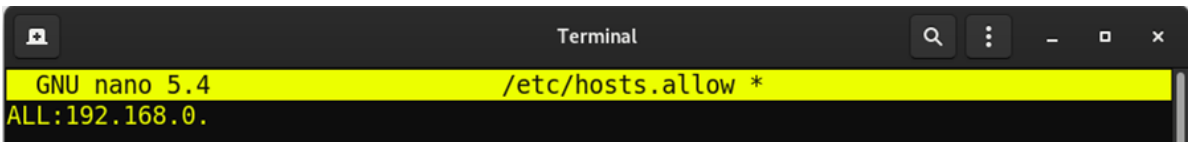


The nameserver line provides the IP addresses of name servers for your domain. If you have multiple name servers, list them on separate lines. They're queried in the order in which they appear in the file.
/etc/hosts.allow on a Linux system

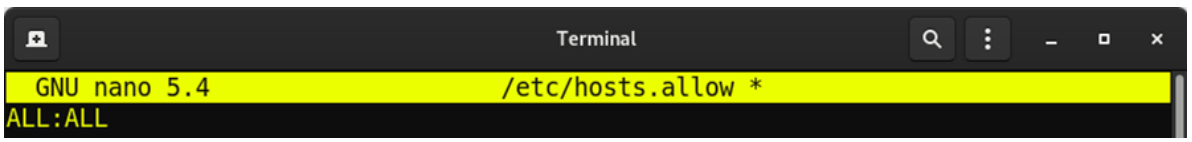
5) /etc/hosts.allow on a Linux system

The /etc/hosts.allow file specifies which hosts are allowed to use the Internet services (such as Telnet and FTP) running on your system. This file is consulted before certain Internet services start. The services start only if the entries in the hosts.allow file imply that the requesting host is allowed to use the services.

If you want to let all local hosts have access to all Internet services, you can use the ALL keyword and rewrite the line as follows:



Finally, to open all Internet services to all hosts, you can replace the IP address with ALL, as follows:



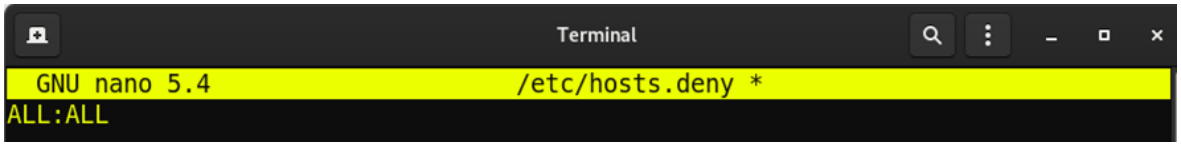
You can also use host names in place of IP addresses.

6) /etc/hosts.deny on a Linux system

The **/etc/hosts.deny** file is the opposite of **/etc/hosts.allow**. Whereas hosts.allow specifies which hosts may access Internet services (such as Telnet and TFTP) on your system, the hosts.deny file identifies the hosts that must be denied services.

The /etc/hosts.deny file is consulted if no rules in the /etc/hosts.allow file apply to the requesting host. Service is denied if the hosts.deny file has a rule that applies to the host.
The entries in **/etc/hosts.deny** file have the same format as those in the **/etc/hosts.allow** file; they're in server: IP address format, where server refers to the name of the program providing a specific Internet service and IP address identifies the host that must not be allowed to use that service.

If you already set up entries in the /etc/hosts.allow file to allow access to specific hosts, you can place the following line in /etc/hosts.deny to deny all other hosts access to any service on your system:

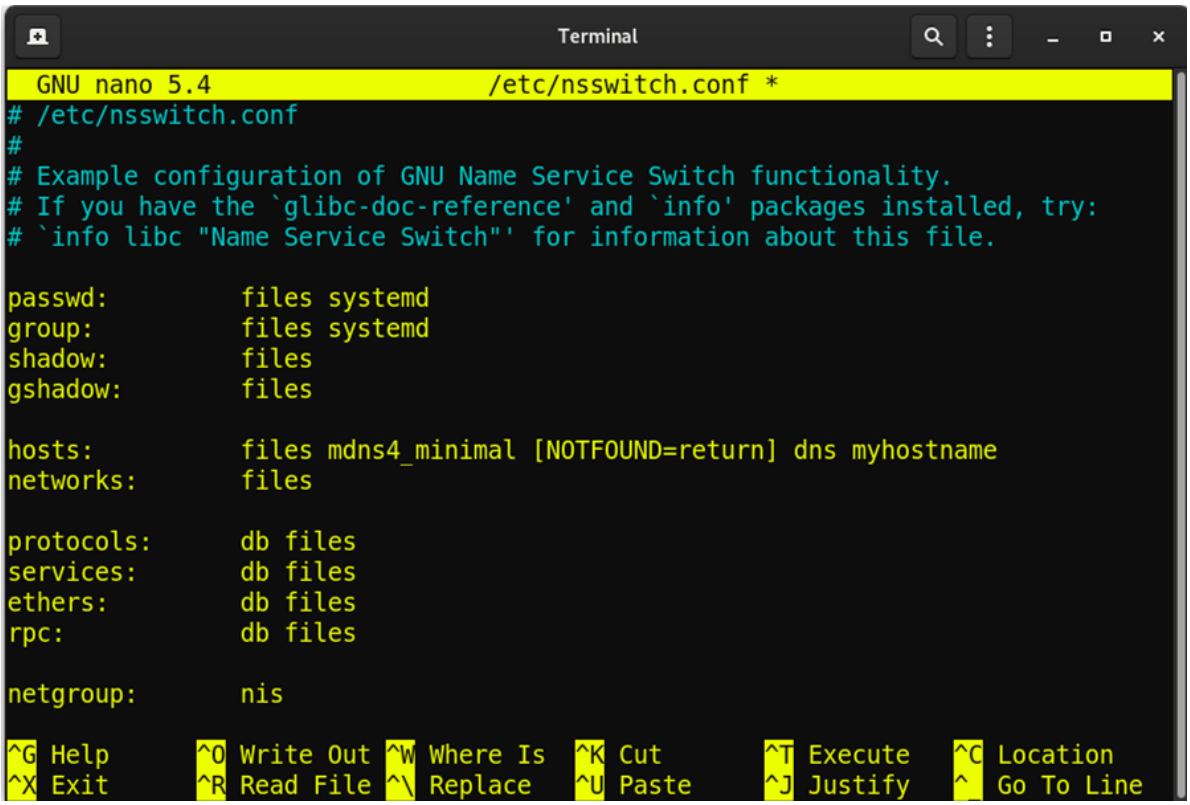


7) /etc/nsswitch.conf on a Linux system

The /**etc/nsswitch.conf** file, known as the **name service switch (NSS)** file, specifies how services such as the resolver library, NIS, NIS+, and local configuration files (such as /etc/hosts and /etc/shadow) interact.

NIS and NIS+ are network information systems — another type of name-lookup service. Newer versions of the Linux kernel use the /etc/nsswitch.conf file to determine what takes precedence: a local configuration file, a service such as DNS (Domain Name System), or NIS.

As an example, the following hosts entry in the /etc/nsswitch.conf file says that the resolver library first tries the /etc/hosts file, then tries NIS+, and finally tries DNS:



6.1 C : Network Interfaces and Routes: ifconfig, route, ping, netstat, tcpdump commands.

1) ifconfig

The ifconfig tool allows us to configure our network interfaces, if we don't have any network interfaces set up, the kernel's device drivers and the network won't know how to talk to each other. Ifconfig runs on bootup and configures our interfaces through config files, but we can also manually modify them. The output of ifconfig shows the interface name on the left side and the right side shows detailed information. You'll most commonly see interfaces named eth0 (first Ethernet card in the machine), wlan0 (wireless interface), lo (loopback interface). The loopback interface is used to represent your computer, it just loops you back to yourself. This is good for debugging or connecting to servers running locally.

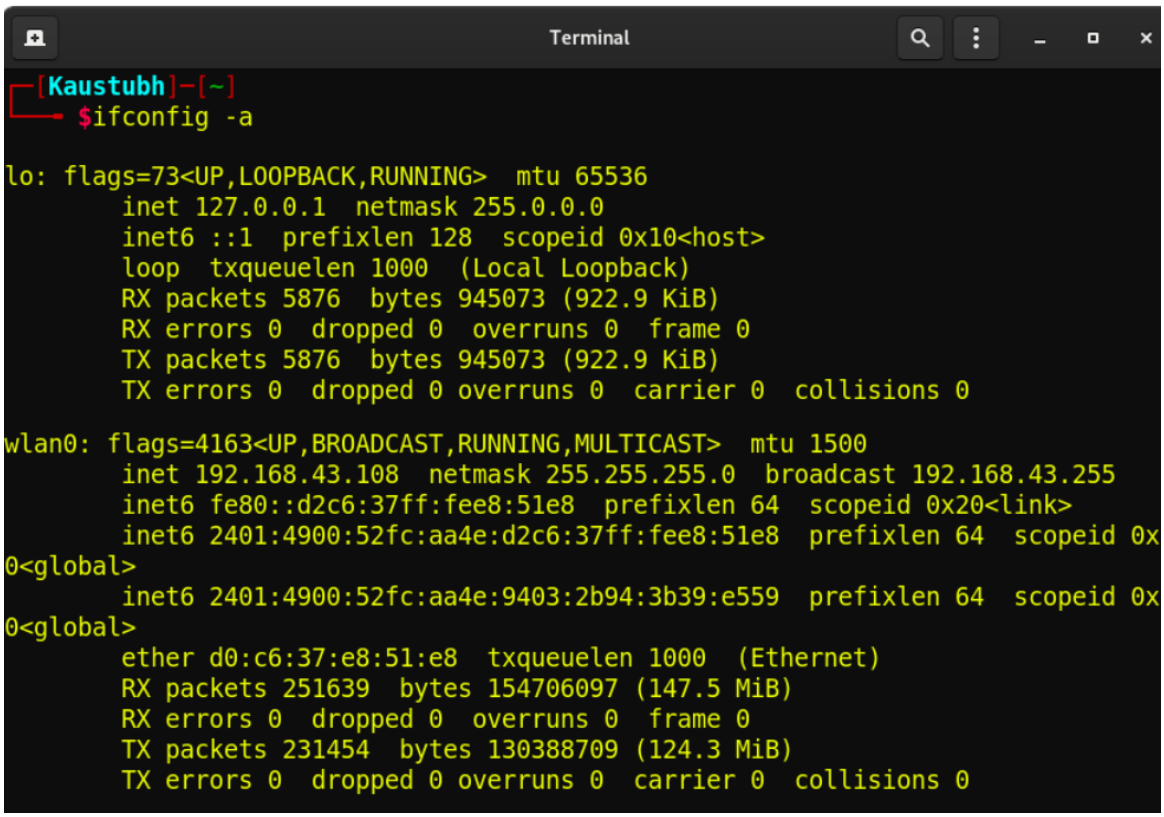
The status of interfaces, can be up or down, as you can guess if you wanted to "turn off" an interface you can set it to go down. The fields you'll probably look at the most in the ifconfig output is the HWaddr (MAC address of the interface), inet address (IPv4 address) and inet6 (IPv6 address). Of course you can see that the subnet mask and broadcast address are there as well. You can also view interface information at

/etc/network/interfaces.

Viewing the configuration of all interfaces

If you'd like to view the configuration of all network interfaces on the system (not just the ones that are currently active), you can specify the -a option, like this:

```
sudo ifconfig -a
```

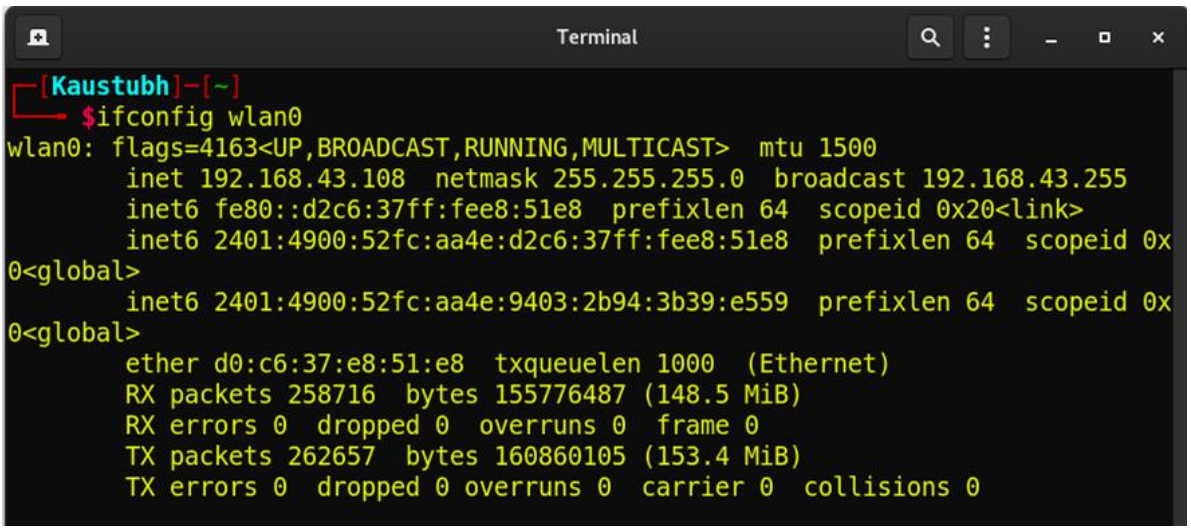


This produces output similar to running ifconfig, but if there are any inactive interfaces on the system, their configuration is also shown.

Viewing the configuration of a specific interface

To view the configuration of a specific interface, specify its name as an option. For instance,

```
sudo ifconfig wlan0
```



displays the configuration of device wlan0 only.

Enabling and disabling an interface

When a network interface is active, it can send and receive data; when it is inactive, it is not able to transmit or receive. You can use ifconfig to change the status of a network interface from inactive to active, or vice versa.

To enable an inactive interface, provide ifconfig with the interface name followed by the keyword up.

Enabling or disabling a device requires superuser permissions, so you either have to be logged in as root, or prefix your command with sudo to run it with superuser privileges.

For instance, if network interface wlan0 is inactive, you can activate it with the command:

```
sudo ifconfig wlan0 up
```

Similarly, you can disable an active network interface using the down keyword. For

instance, to disable the wireless network interface wlan0, use the command:

```
sudo ifconfig wlan0 down
```

Configuring an interface

ifconfig can be used at the command line to configure (or re-configure) a network interface.

To assign a static IP address to an interface, specify the interface name and the IP address. For example, to assign the IP address 192.168.43.108 to the interface wlan0, use the command:

```
sudo ifconfig wlan0 192.168.43.108
```

To assign a network mask to an interface, use the keyword netmask and the netmask address. For instance, to configure the interface wlan0 to use a network mask of 255.255.255.0, the command would be:

```
sudo ifconfig wlan0 255.255.255.0
```

To assign a broadcast address to an interface, use the keyword broadcast and the broadcast address. For instance, to configure the interface wlan0 to use a broadcast address of 192.168.43.255, the command would be:

```
sudo ifconfig wlan0 broadcast 192.168.43.255
```

These configurations can combined in a single command. For instance, to configure interface eth0 to use the static IP address 192.168.43.108, the network mask 255.255.255.0, and the broadcast address 192.168.43.255, the command would be:

```
sudo ifconfig wlan0 192.168.43.108 netmask 255.255.255.0 broadcast 192.168.43.255
```

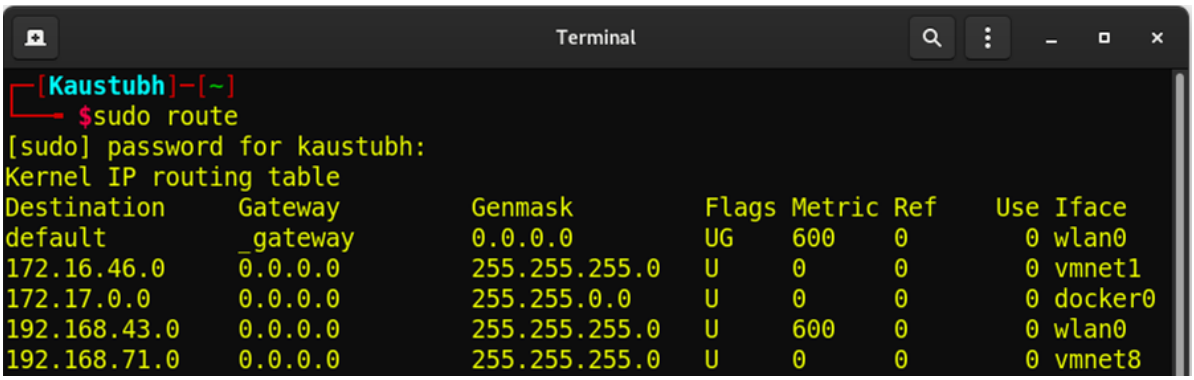
2) route

In computer networking, a router is a device responsible for forwarding network traffic. When datagrams arrive at a router, the router must determine the best way to route them to their destination.

On Linux, BSD, and other Unix-like systems, the route command is used to view and make changes to the kernel routing table. The command syntax is different on different systems; here, with specific command syntax, we'll be discussing the Linux version.

Running route at the command line without any options displays the routing table entries:

```
sudo route
```



This shows us how the system is currently configured. If a packet comes into the system and has a destination in the range 192.168.43.0 through 192.168.43.255, then it is forwarded to the gateway, which is 0.0.0.0 — a special address which represents an invalid or non-existant destination. So, in this case, our system will not route these packets.

Examples:

```
sudo route -n
```

Showing routing table for all IPs bound to the server.

```
sudo route add -net 192.56.76.0 netmask 255.255.255.0 dev eth0
```

adds a route to the network 192.56.76.x via "eth0" The Class C netmask modifier is not really necessary here because >192.* is a Class C IP address. The word "dev" can be omitted here.

```
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev eth0
```

This command sets all of the class D (multicast) IP routes to go via "eth0". This is the correct normal configuration for a multicasting kernel.

3) ping

ping is a simple way to send network data to, and receive network data from, another computer on a network. It is frequently used to test, at the most basic level, whether another system is reachable over a network, and if so, how much time it takes for that data to be exchanged.

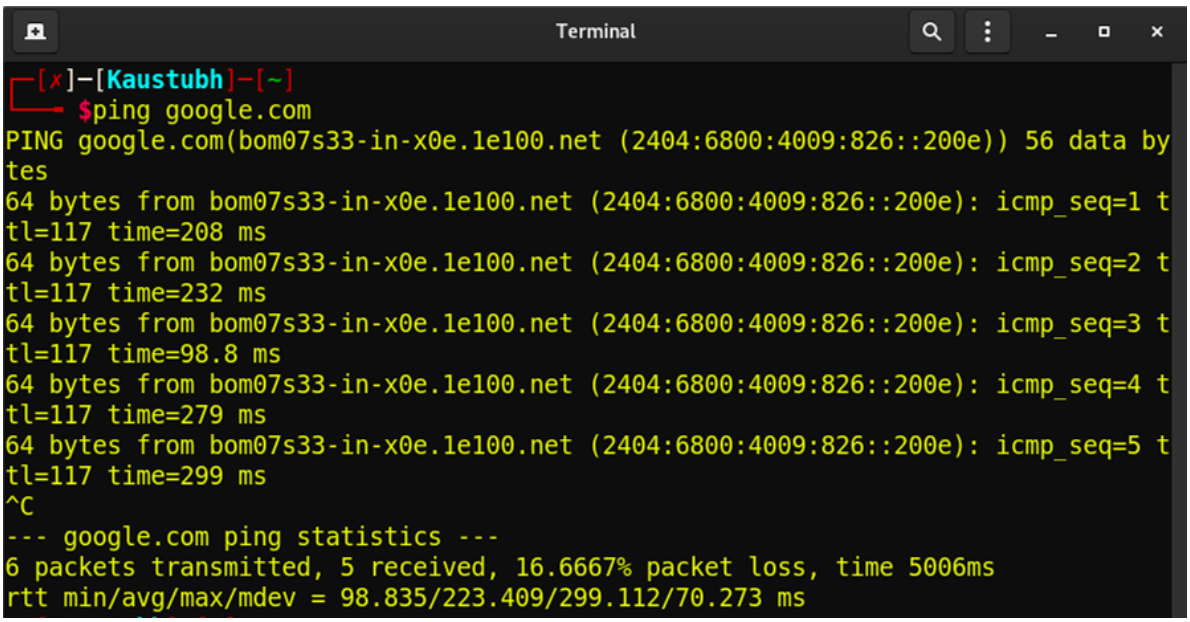
The ping utility uses the ICMP protocol's mandatory ECHO_REQUEST datagram to elicit an ICMP ECHO_RESPONSE from a host or gateway. ECHO_REQUEST datagrams ("pings") have an IP and ICMP header, followed by a struct timeval and then an arbitrary number of "pad" bytes used to fill out the packet.

ping is intended for use in network testing, measurement and management. Because of the load it can impose on the network, it is unwise to use ping during normal operations or from automated scripts.

Examples:

Ping the host google.com to see if it's alive.

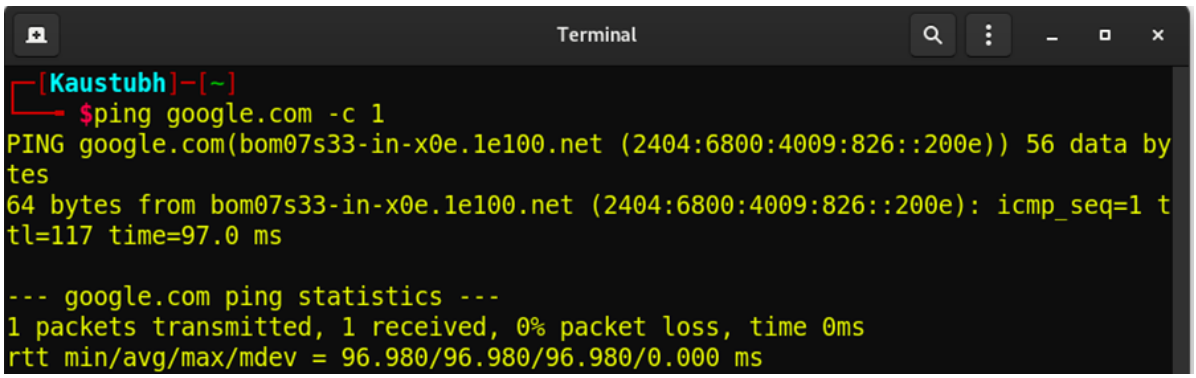
```
ping google.com
```



Ping the host google.com once.

```
ping google.com -c 1
```

Output resembles the following:



4) netstat

netstat ("network statistics") is a command-line tool that displays network connections (both incoming and outgoing), routing tables, and many network interface (network interface controller or software-defined network interface) and network protocol statistics. It is available on Unix-like operating systems, including OS X, Linux, Solaris, and BSD, and on Windows NT-based operating systems, including Windows XP, Windows Vista, Windows 7 and Windows 8.

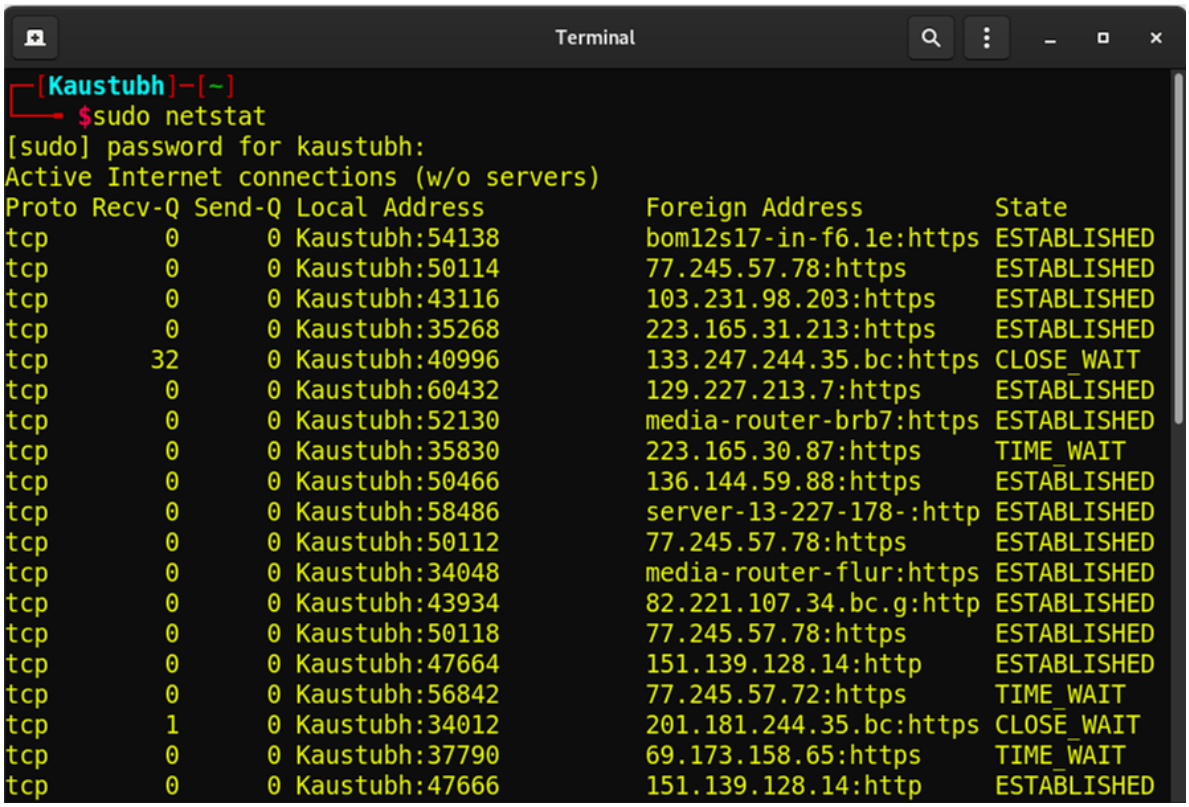
It is used for finding problems in the network and to determine the amount of traffic on the network as a performance measurement.

Examples:

```
sudo netstat
```

Displays generic statistics about the network activity of the local system.

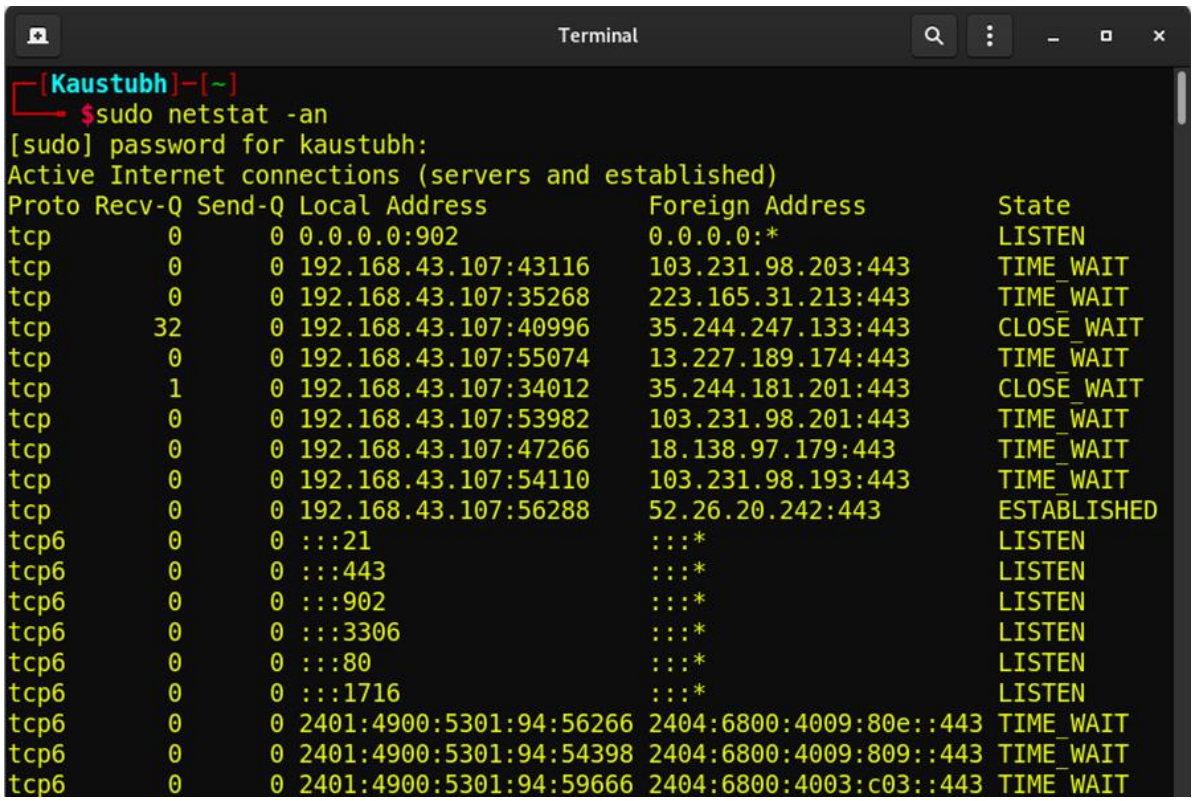
Sample output:




```
sudo netstat -an
```

Shows information about all active connections to the server, including the source and destination IP addresses and ports, if you have proper permissions.

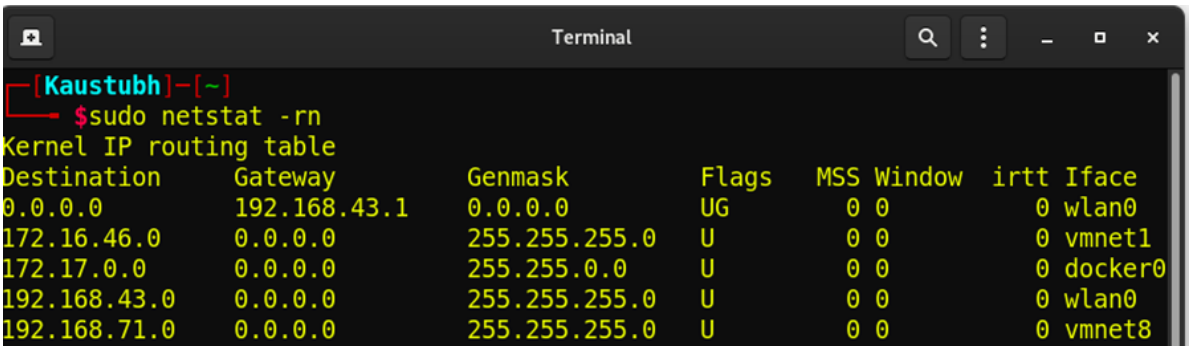
Sample output:




```
sudo netstat -rn
```

Displays the routing table for all IP addresses bound to the server.

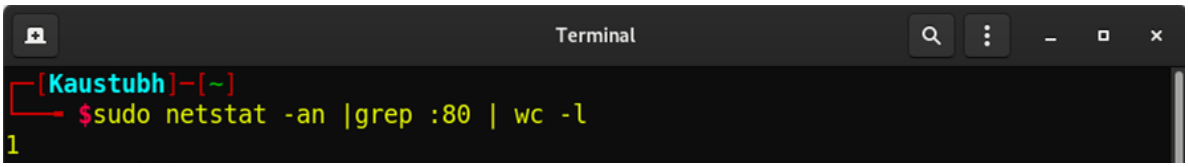
Sample output:




```
sudo netstat -an |grep :80 | wc -l
```

Collects statistics about the amount of active connections on port 80, and pipes this data to the wc command, which displays the number of connections by counting the lines of the original netstat output.

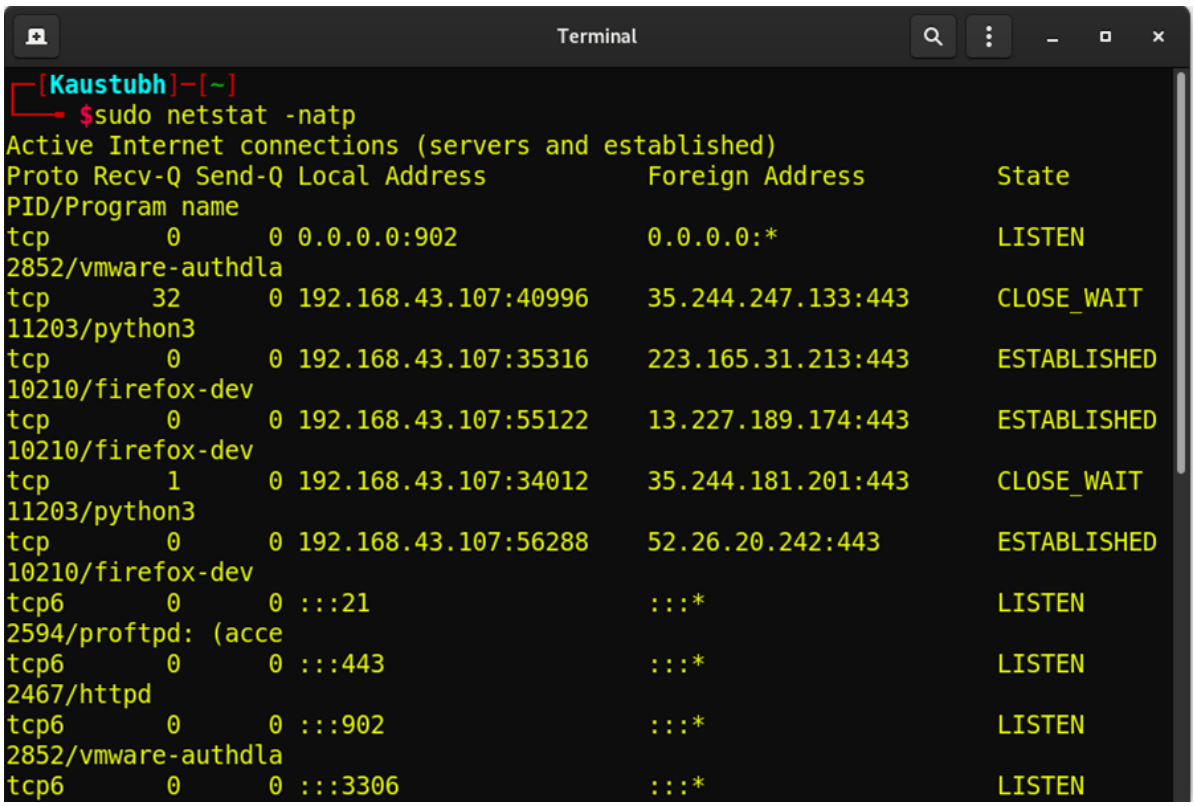
Sample output:



sudo netstat -natp

Display statistics about active Internet connections.

Sample output:



5) tcpdump

Tcpdump prints out a description of the contents of packets on a network interface that match the boolean expression specified on the command line. It can also be run with the -w flag, which causes it to save the packet data to a file for later analysis, or with the -r flag, which causes it to read from a saved packet file rather than to read packets from a network interface.

Tcpdump will, if not run with the -c flag, continue capturing packets until it is interrupted by a SIGINT signal (for example, when the user types the interrupt character, typically control-C) or a SIGTERM signal (typically generated with the kill command); if run with the -c flag, it will capture packets until it is interrupted by a SIGINT or SIGTERM signal or the specified number of packets have been processed.

Reading packets from a network interface may require that you have special privileges; see the pcap (3PCAP) manual for details. Reading a saved packet file doesn't require special privileges.

Examples:

```
sudo tcpdump host sundown
```

Prints all packets arriving at or departing from host sundown.

```
sudo tcpdump host helios and \( hot or ace \)
```

Prints traffic between host helios and either hot or ace.

```
sudo tcpdump ip host ace and not helios
```

Prints all IP packets between ace and any host except helios.

```
sudo tcpdump 'gateway snup and (port ftp or ftp-data)'
```

Prints all ftp traffic through Internet gateway snup. Note that the expression is quoted to prevent the shell from interpreting the parentheses.

```
sudo tcpdump ip and not net localnet
```

Prints traffic neither sourced from nor destined for local hosts. If you gateway to another network, this stuff should never make it onto your local network.

```
tcpdump 'tcp[tcpflags] & (tcp-syn|tcp-fin) != 0 and not src and dst net localnet'
```

Prints all IPv4 HTTP packets to and from port 80. tcpdump will print only packets that contain data; not, for example, SYN and FIN packets and ACK-only packets.

```
tcpdump 'gateway snup and ip[2:2] > 576'
```

Prints IP broadcast or multicast packets that were not sent via Ethernet broadcast or multicast.

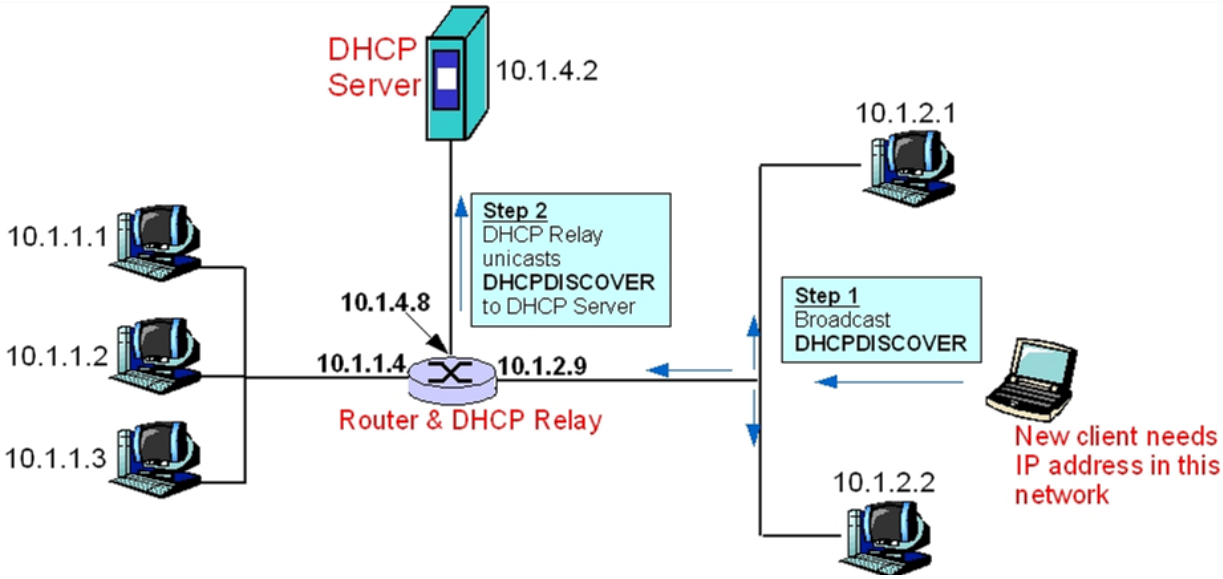
```
tcpdump 'icmp[icmptype] != icmp-echo and icmp[icmptype] != icmp-echoreply'
```

Prints all ICMP packets that are not echo requests/replies (i.e. not ping packets).

6.2 : DHCP Configuration: Configuring DHCP Client and Server, Dynamic Address, Fixed Addresses.

DHCP:

Dynamic Host Configuration Protocol is a network management protocol that is used to dynamically assign the IP address and other information to each host on the network so that they can communicate efficiently. DHCP automates and centrally manages the assignment of IP address easing the work of network administrator. In addition to the IP address, the DHCP also assigns the subnet masks, default gateway and domain name server(DNS) address and other configuration to the host and by doing so, it makes the task of network administrator easier.



Components of DHCP

- **DHCP Server**: It is typically a server or a router that holds the network configuration information.

- **DHCP Client:** It is the endpoint that gets the configuration information from the server like any computer or mobile.

- **DHCP Relay Agent:** If you have only one DHCP Server for multiple LAN's then the DHCP relay agent present in every network will forward the DHCP request to the servers. This because the DHCP packets cannot travel across the router. Hence, the relay agent is required so that DHCP servers can handle the request from all the

networks.

- **IP address pool:** It contains the list of IP address which are available for assignment to the client.

- **Subnet Mask:** It tells the host that in which network it is currently present.

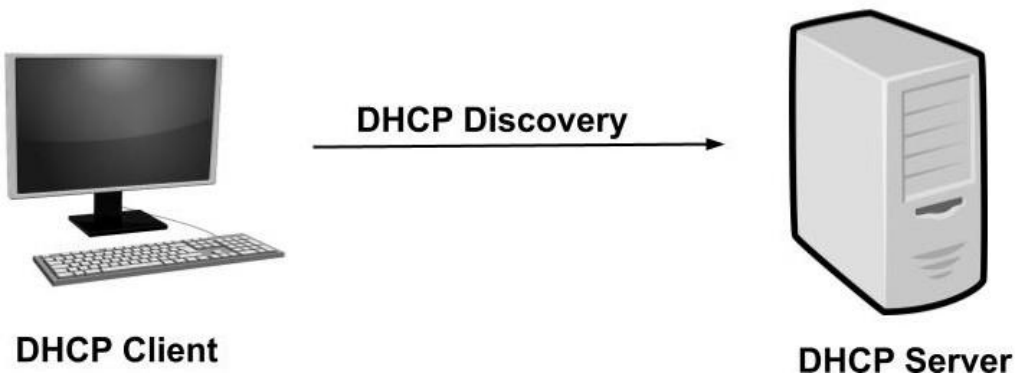
- **Lease Time:** It is the amount of time for which the IP address is available to the client. After this time the client must renew the IP address.

- **Gateway Address:** The gateway address lets the host know where the gateway is to connect to the internet.

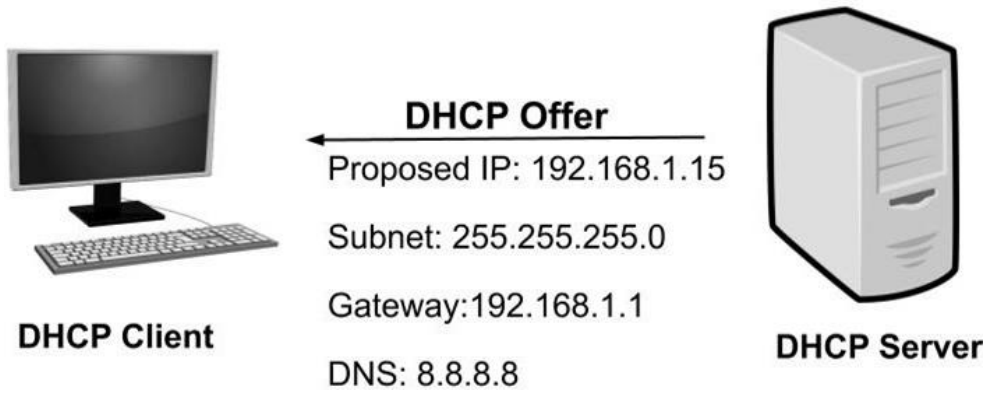
Working of DHCP:

DHCP works at the application layer to dynamically assign the IP address to the client and this happens through the exchange of a series of messages called DHCP transactions or DHCP conversation.

DHCP Discovery: The DHCP client broadcast messages to discover the DHCP servers. The client computer sends a packet with the default broadcast destination of 255.255.255.255 or the specific subnet broadcast address if any configured. 255.255.255.255 is a special broadcast address, which means "this network": it lets you send a broadcast packet to the network you're connected to.

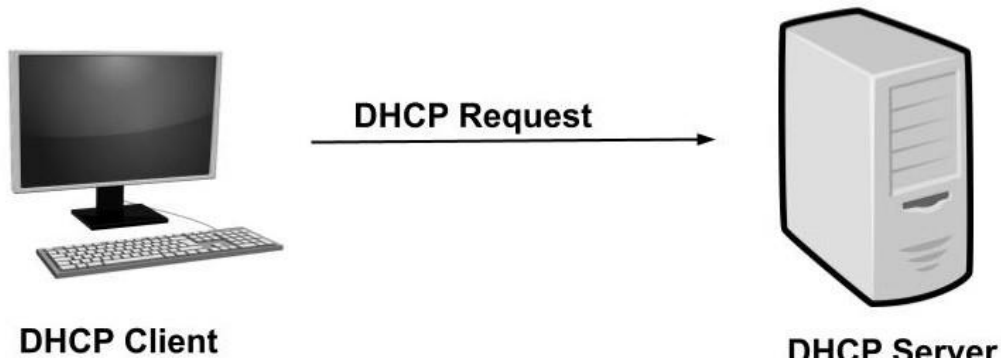


DHCP Offer: When the DHCP server receives the DHCP Discover message then it suggests or offers an IP address(form IP address pool) to the client by sending a DHCP offer message to the client. This DHCP offer message contains the proposed IP address for DHCP client, IP address of the server, MAC address of the client, subnet mask, default gateway, DNS address, and lease information.

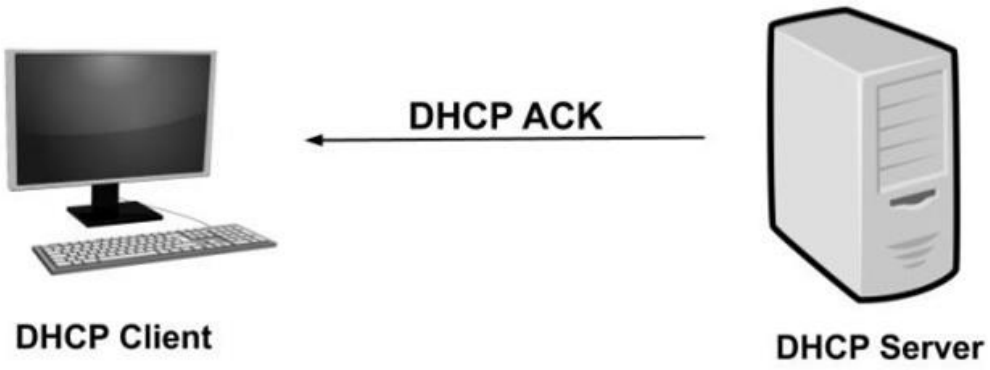


1. the proposed IP address for DHCP client (here 192.168.1.11)
2. Subnet mask to identify the network (here 255.255.255.0)
3. IP of the default gateway for the subnet (here 192.168.1.1)
4. IP of DNS server for name translations (here 8.8.8.8)

DHCP Request: In most cases, the client can receive multiple DHCP offer because in a network there are many DHCP servers(as they provide fault tolerance). If the IP addressing of one server fails then other servers can provide backup. But, the client will accept only one DHCP offer. In response to the offer, the client sends a DHCP Request requesting the offered address from one of the DHCP servers. All the other offered IP addresses from remaining DHCP servers are withdrawn and returned to the pool of IP available addresses.



DHCP Acknowledgment: The server then sends Acknowledgment to the client confirming the DHCP lease to the client. The server might send any other configuration that the client may have asked. At this step, the IP configuration is completed and the client can use the new IP settings.



IP Settings

IP: 192.168.1.15

Subnet: 255.255.255.0

Default Gateway:192.168.1.1

DNS: 8.8.8.8

Advantages of DHCP

- It is easy to implement and automatic assignment of an IP address means an accurate IP address.
- The manual configuration of the IP address is not required. Hence, it saves time and workload for the network administrators.
- Duplicate or invalid IP assignments are not there which means there is no IP address conflict.
- It is a great benefit for mobile users as the new valid configurations are automatically obtained when they change their network.

Disadvantages of DHCP

- As the DHCP servers have no secure mechanism for the authentication of the client so any new client can join the network. This posses security risks like unauthorized clients being given IP address and IP address depletion from unauthorized clients.
- The DHCP server can be a single point of failure if the network has only one DHCP server.

Installing DHCP Server

The DCHP server package is available in the official repositories of mainstream Linux

distributions, installing is quite easy, simply run the following command.

```
sudo apt-get install isc-dhcp-server
```

Once the installation is complete, configure the interface on which you want the **DHCP** daemon to serve requests in the configuration file **/etc/default/isc-dhcp-server** or **/etc/sysconfig/dhcpd**.

```
sudo vim /etc/default/isc-dhcp-server
```

For example, if you want the **DHCPD** daemon to listen on `eth0`, set it using the following directive.

```
DHCPDARGS="eth0"
```

Configuring DHCP Server for dynamic addresses

The main DHCP configuration file is located at **/etc/dhcp/dhcpd.conf**, which should contain settings of what to do, where to do something and all network parameters to provide to the clients.

This file basically consists of a list of statements grouped into two broad categories:

- **Global parameters:** specify how to carry out a task, whether to carry out a task, or what network configuration parameters to provide to the DHCP client.
- **Declarations:** define the network topology, state a clients is in, offer addresses for the clients, or apply a group of parameters to a group of declarations.

Now, open and edit the configuration file to configure your DHCP server.

```
sudo vim /etc/dhcp/dhcpd.conf
```

Start by defining the **global parameters** which are common to all supported networks, at the top of the file. They will apply to all the declarations:

```
option domain-name "tecmint.lan";
option domain-name-servers ns1.tecmint.lan, ns2.tecmint.lan;
default-lease-time 3600;
max-lease-time 7200;
authoritative;
```

Next, you need to define a sub-network for an internal subnet i.e **192.168.1.0/24** as shown.

```
subnet 192.168.1.0 netmask 255.255.255.0 {
        option routers                  192.168.1.1;
        option subnet-mask              255.255.255.0;
        option domain-search            "tecmint.lan";
        option domain-name-servers      192.168.1.1;
        range   192.168.10.10   192.168.10.100;
        range   192.168.10.110   192.168.10.200;
}
```

Note that hosts which require special configuration options can be listed in host statements (see the dhcpd.conf man page).

Now that you have configured your DHCP server daemon, you need to start the service for the mean time and enable it to start automatically from the next system boot, and check if its up and running using following commands.

```
$ sudo systemctl start isc-dhcp-server
$ sudo systemctl enable isc-dhcp-server
$ sudo systemctl enable isc-dhcp-server
```

Next, permit requests to the DHCP daemon on Firewall, which listens on port **67/UDP**, by running.

```
$ sudo ufw allow 67/udp
$ sudo ufw reload
```

Configuring DHCP Server for fixed addresses

Everything needs to be same like we configured DHCP server for dynamic addresses. Only if you want to give a fixed IP address to any particular machine in a network in that case you have to add following lines in your **/etc/dhcp/dhcpd.conf** file.

```
host windows{
        hardware ethernet 52:54:00:da:dd:13;
        fixed-address 192.168.1.2;
}
```

In the above configuration **windows** is the hostname, **52:54:00:da:dd:13** is the MAC address of that machine and **192.168.1.2** is fixed IP address provided to that machine.

Now restart your DHCP server.

```
$ sudo systemctl start isc-dhcp-server
```

Configuring DHCP Clients

Finally, you need to test if the DHCP server is working fine. Logon to a few client machines on the network and configure them to automatically receive IP addresses from the server.

Modify the appropriate configuration file for the interface on which the clients will auto-receive IP addresses.

you can configure all interface in the config file **/etc/network/interfaces**.

```
sudo vi /etc/network/interfaces
```

Add these lines in it:

```
auto  eth0
iface eth0 inet dhcp
```

Save the file and restart network services (or reboot the system).

```
sudo systemctl restart networking
```